INSTITUTO SUPERIOR TECNOLÓGICO DE LA JOYA

TALLER DE LENGUAJES DE PROGRAMACIÓN

INFORME DE LA GUÍA DE APRENDIZAJE N° 3

POLIMORFISMO Y CLASES ABSTRACTAS EN JAVA

Estudiante: Marcos Riquelme huahuiza Sarayasi

Proyecto base: GameStore Escolar

1. Introducción

En esta sesión trabajé con el proyecto base GameStore Escolar, que es una tienda de recompensas. Lo que tenía que hacer era programar la parte lógica usando clases abstractas, herencia y polimorfismo. El programa es de consola, o sea que no tiene ventanas y todo se ve en la terminal.

Lo hice en cuatro archivos de Java, uno por cada clase, porque en Java cada clase pública debe estar en un archivo con su mismo nombre. Al final compilé todo y comprobé que el programa mostrara mensajes distintos para cada tipo de ítem.

2. Objetivo

Programar la lógica de negocio orientada a objetos creando una clase abstracta, aplicando herencia con subclases y usando polimorfismo para manejar distintos tipos de recompensas desde una misma referencia.

3. Herramientas que utilicé

JDK (Java Development Kit): me da el compilador `javac` y la máquina virtual de Java para ejecutar el programa.

Visual Studio Code: el editor donde escribí las clases, con la extensión de Java instalada.

Terminal de PowerShell: la que viene dentro de VS Code, donde compilé y ejecuté.

La carpeta de trabajo fue `D:\Taller_de_Lenguaje_de_Programacion\Sesion_3\codigo`.

4. Marco teórico breve

Clase abstracta. Es una clase que no se puede instanciar directamente, o sea que no puedo escribir `new ItemRecompensa(...)`. Sirve como plantilla para otras clases. Puede tener métodos normales y también métodos abstractos, que son métodos sin cuerpo que las clases hijas están obligadas a escribir.

Herencia. Es cuando una clase hija toma los atributos y métodos de una clase madre usando la palabra `extends`. Así no se repite código.

Polimorfismo. Es la capacidad de que objetos de distintas clases respondan al mismo método de manera diferente. Se logra sobrescribiendo métodos con `@Override` y guardando los objetos en una referencia común, como un arreglo del tipo de la clase padre.

super(...). Se usa en el constructor de la clase hija para llamar al constructor de la clase madre. Debe ir en la primera línea.

5. Desarrollo

5.1 Clase madre abstracta: ItemRecompensa

Esta es la base de todo. Tiene dos atributos privados, `nombre` y `costoPuntos`, un constructor para guardarlos y dos getters para poder leerlos desde afuera. Lo más importante es el método `aplicarEfecto()`, que es abstracto: no tiene cuerpo, solo dice que cualquier ítem debe saber aplicar su efecto, pero no dice cómo. Eso lo decide cada subclase.

```java
// Clase Madre Abstracta (Trabajo 1)
public abstract class ItemRecompensa {
    private String nombre;
    private int costoPuntos;

    public ItemRecompensa(String nombre, int costoPuntos) {
        this.nombre = nombre;
        this.costoPuntos = costoPuntos;
    }

    // Método abstracto que las subclases deben implementar
obligatoriamente
    public abstract void aplicarEfecto();

    // Métodos Getters
    public String getNombre() { return nombre; }
    public int getCostoPuntos() { return costoPuntos; }
}
```

5.2 Subclase 1: ItemAvatar

Esta clase hereda de ItemRecompensa con extends. Agrega un atributo propio, tipoSkin. En su constructor llamo a super(nombre, costoPuntos) para que la clase madre guarde el nombre y el costo, y después guardo el tipo de skin. Sobrescribí aplicarEfecto() con @Override para que imprima que se está desbloqueando una skin de avatar.

```java
/// Subclase 1: ItemAvatar (Trabajo 1)
public class ItemAvatar extends ItemRecompensa {
    private String tipoSkin;

    public ItemAvatar(String nombre, int costoPuntos, String tipoSkin) {
        super(nombre, costoPuntos);
        this.tipoSkin = tipoSkin;
    }

    @Override
    public void aplicarEfecto() {
        System.out.println("Desbloqueando skin de avatar tipo: " +
tipoSkin);
    }
}
```

5.3 Subclase 2: ItemPaseAcademico

Funciona igual que la anterior, pero con su propio atributo, duracionDias. Su versión de aplicarEfecto() imprime que se otorga un pase académico especial por una cantidad de días.

```java
// Subclase 2: ItemPaseAcademico (Trabajo 1)
public class ItemPaseAcademico extends ItemRecompensa {
    private int duracionDias;

    public ItemPaseAcademico(String nombre, int costoPuntos, int
duracionDias) {
        super(nombre, costoPuntos);
        this.duracionDias = duracionDias;
    }

    @Override
    public void aplicarEfecto() {
        System.out.println("Otorgando pase académico especial por " +
duracionDias + " días.");
    }
}
```

5.4 Clase de prueba: PrincipalPrueba

Esta es la clase que tiene el método main, o sea la que se ejecuta. Aquí hice un arreglo de tipo ItemRecompensa con dos posiciones. En la primera guardé un ItemAvatar (Skin Guerrero Cyber, 150 puntos, tipo Legendario) y en la segunda un ItemPaseAcademico (Pase de Prórroga, 300 puntos, 7 días).

Después recorrí el arreglo con un for y, para cada ítem, imprimí su nombre y costo y llamé a aplicarEfecto(). Aunque el for usa la misma referencia ItemRecompensa para todos, cada objeto ejecuta su propia versión del método. Eso es el polimorfismo.

```java
/*
 * SUSTENTACIÓN CONCEPTUAL
 * 1) ¿Por qué no se puede instanciar una clase abstracta?
 *    Porque ItemRecompensa es solo una plantilla: tiene un método
 *    (aplicarEfecto) sin cuerpo, así que un objeto directo de ella
 *    no sabría qué hacer. Solo se crean objetos de subclases concretas
 *    como ItemAvatar o ItemPaseAcademico.
 * 2) ¿Qué ventaja aporta @Override?
 *    Le avisa al compilador que el método sobrescribe uno del padre.
 *    Si escribo mal el nombre, el compilador marca error en vez de
 *    crear un método nuevo por accidente.
 */
public class PrincipalPrueba {
    public static void main(String[] args) {
        // Creación de un arreglo polimórfico de tipo ItemRecompensa
(Trabajo 2)
        ItemRecompensa[] inventarioTienda = new ItemRecompensa[2];

        inventarioTienda[0] = new ItemAvatar("Skin Guerrero Cyber", 150,
"Legendario");
```

```
        inventarioTienda[1] = new ItemPaseAcademico("Pase de Prórroga",
300, 7);

        System.out.println("--- EJECUTANDO EFECTOS CON POLIMORFISMO ---");

        // Recorriendo el arreglo e invocando el método de forma
polimórfica (Trabajo 2)
        for (ItemRecompensa item : inventarioTienda) {
            System.out.println("Ítem: " + item.getNombre() + " | Costo: " +
item.getCostoPuntos() + " pts");
            item.aplicarEfecto(); // Comportamiento dinámico según la
subclase
            System.out.println("----------------------------------------
--------");
        }
    }
}
```

6. Compilación y ejecución

Abrí la terminal de VS Code dentro de la carpeta codigo y usé estos dos comandos:

javac *.java compila los cuatro archivos a la vez. Es importante compilarlos juntos porque PrincipalPrueba depende de las otras tres clases. Si salió bien, no muestra ningún mensaje y se generan los archivos .class.

java PrincipalPrueba ejecuta el programa. Se escribe sin .java ni .class.

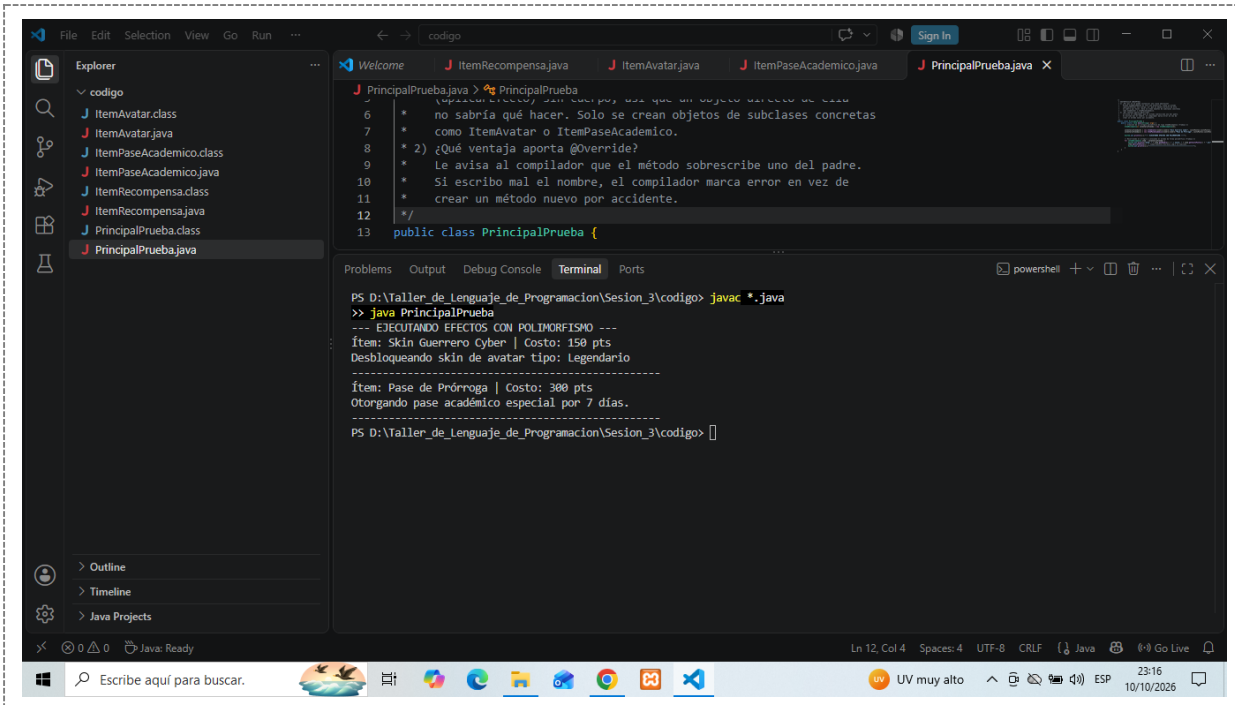
Al principio intenté escribir cd Sesion-03/codigo como aparecía en el manual, pero me dio error de ruta porque mi terminal ya estaba dentro de la carpeta codigo. Entendí que ese comando era solo un ejemplo del manual y que en mi computadora no hacía falta.

7. Resultado obtenido

Al ejecutar java PrincipalPrueba el programa mostró lo siguiente:

```
--- EJECUTANDO EFECTOS CON POLIMORFISMO ---
Ítem: Skin Guerrero Cyber | Costo: 150 pts
Desbloqueando skin de avatar tipo: Legendario
--------------------------------------------------
Ítem: Pase de Prórroga | Costo: 300 pts
Otorgando pase académico especial por 7 días.
--------------------------------------------------
```

Captura de la ejecución:



Como se ve, el primer ítem imprime el mensaje de ItemAvatar y el segundo el de ItemPaseAcademico. Los mensajes son diferentes aunque se llamó al mismo método, y eso confirma que el polimorfismo funciona.

8. Sustentación conceptual

¿Por qué no se puede instanciar directamente una clase abstracta? Porque ItemRecompensa es solo una plantilla. Tiene un método sin cuerpo, aplicarEfecto(), y un objeto creado directamente de ella no sabría qué hacer al llamarlo. Por eso Java no lo permite y solo se pueden crear objetos de las subclases concretas, como ItemAvatar o ItemPaseAcademico.

¿Qué ventaja aporta @Override? Le indica al compilador que el método está sobrescribiendo uno de la clase padre. Si me equivoco al escribir el nombre, el compilador marca error en vez de crear un método nuevo sin que yo me dé cuenta.

9. Dificultades encontradas

La principal fue el error del cd que mencioné arriba. Además, aprendí que hay que compilar todos los archivos juntos, porque si compilo solo PrincipalPrueba.java aparece el error "cannot find symbol". También tuve que acordarme de poner super(...) como primera línea en los constructores de las subclases, ya que la clase madre no tiene un constructor vacío.

10. Conclusiones

Una clase abstracta me permite definir qué debe hacer un objeto sin decir cómo, y obliga a las subclases a completarlo.

La herencia evita repetir código, porque nombre y costoPuntos se escriben una sola vez en la clase madre.

Con el polimorfismo puedo guardar objetos distintos en un mismo arreglo y tratarlos igual, y cada uno responde a su manera. Esto hace que agregar un nuevo tipo de ítem en el futuro sea fácil, sin cambiar el código del for.

Compilé y ejecuté el programa sin errores y obtuve la salida esperada.